

Product Requirements Document

Signal-Sense: Network Mapper

Category: Social Service Challenge — Intermediate Level Version: 1.0 Status: Draft for Hackathon/Build Planning

1. Executive Summary

Signal-Sense is a two-sided platform that intelligently connects volunteers with local community organizations that need help. Instead of volunteers scrolling through generic listings, an AI-driven recommendation engine surfaces the opportunities most relevant to each person's skills, interests, availability, and location — while organizations get a reliable pipeline of well-matched, motivated volunteers.

The core bet: matching quality drives retention. A volunteer who is recommended a task that fits their skills and schedule is far more likely to show up, do good work, and come back — which is the real bottleneck in most volunteer programs, not sign-up volume.

2. Problem Statement

Community organizations struggle to find volunteers with the right skills at the right time, and often rely on mass emails or generic sign-up sheets with poor targeting. Volunteers, on the other hand, struggle to discover opportunities that genuinely match what they're good at, what they care about, and when they're actually free — leading to mismatched expectations, no-shows, and volunteer burnout or disengagement after one or two tries.

Signal-Sense addresses this as a matching problem, not just a listing problem.

3. Goals & Success Metrics

Goal	Metric
Increase volunteer participation	# active volunteer profiles, # applications submitted/month
Increase volunteer satisfaction & retention	Repeat-volunteering rate, post-task rating avg
Efficient fulfillment for organizations	Time-to-fill per opportunity, fill rate %
Recommendation quality	Click-through rate on recommendations, acceptance rate of recommended matches vs. browsed matches
Community trust	% of organizations verified, report/flag rate

4. User Personas

4.1 Volunteer — "Ananya, the Skilled Helper"

College student or working professional with specific skills (teaching, design, first aid, coding, event management) and limited free time. Wants opportunities that fit her schedule and actually use her skills, not generic "come help set up chairs" tasks — though she's open to those too when nearby and time-boxed.

4.2 Organization Coordinator — "Mr. Verma, NGO Program Lead"

Runs a small-to-mid-size non-profit (education drive, animal shelter, elder care, disaster relief). Time-poor, needs volunteers who show up and are roughly qualified, and doesn't want to manually screen hundreds of unfit applicants.

4.3 Platform Admin — "Trust & Safety"

Verifies organizations before they can post, moderates reports, and monitors platform health metrics.

5. Core Features & Requirements

5.1 Dual Profile System

Volunteer Profile

- Basic info: name, age (13+/18+ tiering if minors allowed), location (geo-coordinates + address)
- Skills (tag-based, multi-select: teaching, medical, IT, design, manual labor, logistics, languages spoken, etc.)
- Interest categories (education, environment, animal welfare, elder care, disaster relief, health camps, etc.)
- Availability (recurring weekly slots + one-off date ranges, calendar-style input)
- Experience/history log (auto-populated from completed opportunities)
- Optional: resume/portfolio upload for skill-verification, certifications (e.g., first-aid certified)

Organization Profile

- Org name, registration/legitimacy documents, category (NGO, school, shelter, government body, community group)
- Location(s) of operation
- Verification status badge
- Points of contact
- Posting history and ratings from past volunteers

5.2 Opportunity Posting (Organization side)

- Title, description, category tags, required skills, # of volunteers needed
- Location (with radius for "remote/flexible" tasks vs. fixed-site tasks)
- Schedule: one-time / recurring / ongoing, with specific date-time slots
- Minimum commitment level (e.g., 2 hrs, one-day, multi-week)
- Application requirements (background check needed, minimum age, etc.)

5.3 Recommendation Engine (AI Core)

Phase 1 — Content-Based Filtering (MVP)

- Vectorize volunteer profile (skills, interests, availability, location) and opportunity listings (required skills, category, location, schedule) into comparable feature vectors, computed in a Java component
- Compute similarity score (cosine similarity on TF-IDF vectors for skills+interests, built with Smile or a hand-rolled Java implementation), combined with a hard/soft filter on geo-distance and schedule overlap
- Rank and surface top-N opportunities per volunteer on their dashboard

Phase 2 — Collaborative Filtering (post-MVP enhancement)

- Once sufficient interaction data exists (applications, completions, ratings), layer in collaborative filtering: "volunteers similar to you also enjoyed X", using Smile's matrix-factorization utilities (or a custom item-based CF implementation in Java)
- Hybrid model: weighted blend of content-based score + collaborative signal, tunable as data volume grows

Ranking factors:

1. Skill/interest match score
2. Geo-distance (closer = higher priority, configurable radius)
3. Schedule compatibility (availability overlap with opportunity time slot)
4. Organization reliability rating
5. Urgency/fill-deadline of the opportunity

5.4 Geo-Location Features

- Map view of nearby opportunities (pin-based, using volunteer's current or home location)
- Distance-based filtering and sorting
- "Opportunities near me this week" quick view
- Optional: heatmap of community need density by area (useful for organizations spotting underserved zones)

5.5 Communication & Scheduling System

- In-app messaging between volunteer and organization coordinator post-application
- Application status tracker (Applied → Under Review → Accepted/Rejected → Completed)
- Automated reminders (push/email/SMS) before scheduled slot
- Calendar sync (Google Calendar/ICS export) for accepted commitments
- Post-completion feedback loop: volunteer rates the org, org rates the volunteer, both feed back into future recommendation quality and trust scores

5.6 Trust & Legitimacy Layer

- Organization verification workflow (document upload — registration certificate/80G/12A or equivalent — reviewed by admin before "Verified" badge is granted)
- Volunteer reporting/flagging system for suspicious postings
- Public rating/review visibility for both sides

6. Tech Stack (Java-Only)

Layer	Technology
Backend framework	Spring Boot (Spring MVC, Spring Web)

Frontend	Thymeleaf templates rendered server-side by Spring Boot (keeps the entire app in one Java codebase)
Database	PostgreSQL + PostGIS, accessed via Spring Data JPA / Hibernate Spatial
Recommendation Engine	Java — Smile (Statistical Machine Intelligence & Learning Engine) or a custom TF-IDF/cosine-similarity implementation using plain Java collections, run as a service class inside the Spring Boot app
Geo-services	Google Maps Services Java Client (geocoding, distance calc, map embeds)
Auth	Spring Security + JWT (JWT-based auth)
Notifications	JavaMail API (email), Firebase Admin SDK for Java (push), Twilio Java SDK (SMS, optional)
Build tool	Maven (or Gradle)
Hosting	Packaged as an executable Spring Boot JAR, deployed via Docker on Render/Railway/AWS Elastic Beanstalk

Keeping the recommendation engine in-process (as a Java service/component) instead of a separate Python microservice avoids cross-language calls and keeps the whole system deployable as a single JAR.

7. Data Model (High-Level)

- User (volunteer): id, profile fields, skills[], interests[], availability[], location, history[]
 - Organization: id, profile fields, verification_status, location(s), ratings
 - Opportunity: id, org_id, title, description, required_skills[], category, location, schedule, slots_available
 - Application: id, user_id, opportunity_id, status, applied_at, completed_at, feedback
 - Recommendation Log: user_id, opportunity_id, score, shown_at, clicked (for model feedback loop)
-

8. Non-Functional Requirements

- Must scale to a two-sided marketplace — cold-start strategy needed for both new volunteers (no history yet) and new orgs (few reviews yet), likely via onboarding surveys weighted heavily in early content-based scoring
 - Data privacy: volunteer PII (especially for minors) must be protected; location data shared only with matched/applied organizations, not publicly
 - Mobile-responsive; majority of volunteer usage likely mobile
 - Recommendation latency should stay under ~1–2 seconds for a snappy dashboard experience
-

9. Risks & Open Conditions

- Two-sided cold start: needs a critical mass of both volunteers and organizations simultaneously to be useful — consider seeding with a manually onboarded partner NGO list at launch
 - Org legitimacy verification: manual review doesn't scale; may need a phased trust model (unverified orgs can post with a visible "unverified" tag while under review, rather than blocking posting entirely)
 - Data sparsity for collaborative filtering: Phase 2 CF won't be reliable until enough interaction data accumulates — content-based filtering should remain the fallback indefinitely for sparse regions/categories
-

10. Potential Dataset

- Kaggle Volunteer Opportunities Dataset — usable for initial EDA and prototyping the content-based recommendation model before real platform data is available.
-

11. Suggested Build Roadmap (Hackathon-Scoped)

1. Day 1: Spring Boot project setup (entities, repositories, Spring Security) + auth + dual profile creation (volunteer & org) forms
2. Day 1–2: Opportunity posting + browsing + basic geo-filter (PostGIS queries via Spring Data JPA)
3. Day 2: Content-based recommendation engine as a Java service (TF-IDF/cosine similarity on skills+interests, with geo/schedule hard filters)
4. Day 2–3: Application flow + status tracker + in-app messaging (basic)
5. Day 3: Polish — map view, notifications, feedback/rating loop, demo prep